COURSE CODE/TITLE: CPE 010-CPE21S1 - Data Structures and Algorithms	**SECTION: CPE21S1**
DATE PERFORMED: 20/10/2026	**DATE SUBMITTED: 22/10/2026**
NAME: Mendoza, Roy Gabriel I. **Pilapil, Loreen Jae D.**	**INSTRUCTOR: Engr. Jimlord Quejado**

ACTIVITY NO. 14p1 Algorithm Complexity

1. PROCEDURE OUTPUT

	Best Case	Worst Case
Algorithm 1: Merge Sort	**O(n log n)**	**O(n log n)**
Algorithm 2: Quick Sort	**O (n log n)**	**O (n^2)**

ANALYSIS

MENDOZA	Merge sort has the same best and worst case because when we do merge sort we divide all the content of a list or an array without any bias, meaning that all the content will be run through and not will be left behind. Then as for the Quick sort, we also divide the array into 2, then work our way using the pivot, as for the worst case, this happens if the pivot is the last element of the sorted array causing all of the contents to be compared with it.
PILAPIL	Merge Sort has O (n log n) as its best and worst case because it divides and merges the given data. On the other hand, Quick sort has O (n log n) as its best case, this happens when the pivot splits the array into two nearly equal halves at every step. It has O (n^2) as its worst case when pivot is consistently the smallest or largest element, producing highly unbalanced partitions

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

CODE

```cpp
//
// Created by royga on 10/20/2026.
//
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>

// MERGE SORT IMPLEMENTATION
template <typename T>
void merge(std::vector<T>& arr, int left, int middle, int right) {
    int size = right - left + 1;
    std::vector<T> temp(size);

    int i = left;
    int j = middle + 1;
    int temp_pos = 0;

    while (i <= middle && j <= right) {
        if (arr[i] <= arr[j]) {
            temp[temp_pos++] = arr[i++];
        } else {
            temp[temp_pos++] = arr[j++];
        }
    }
    while (i <= middle) {
        temp[temp_pos++] = arr[i++];
    }
    while (j <= right) {
        temp[temp_pos++] = arr[j++];
    }
    for (int k = 0; k < size; ++k) {
        arr[left + k] = temp[k];
    }
}
```

```cpp
template <typename T>
void mergeSort(std::vector<T>& arr, int left, int right) {
    if (left >= right) {
        return;
    }
    int middle = left + (right - left) / 2;
    mergeSort([&]arr, left, right: middle);
    mergeSort([&]arr, left: middle + 1, right);
    merge([&]arr, left, middle, right);
}

// QUICK SORT IMPLEMENTATION
template <typename T>
int quick(std::vector<T>& arr, int left, int right) {
    T pivot = arr[right];
    int low = left - 1;
    int high = right;

    while (true) {
        do {
            low++;
        } while (arr[low] < pivot);
        do {
            high--;
        } while (high > left && arr[high] > pivot);

        if (low >= high) {
            break;
        }
        std::swap([&]arr[low], [&]arr[high]);
    }
    std::swap([&]arr[low], [&]arr[right]);
    return low;
}
```

```
72
73  template <typename T>
74  void quickSort(std::vector<T>& arr, int left, int right) {
75      if (left >= right) {
76          return;
77      }
78      int pivotIndex = quick( [&] arr, left, right);
79      quickSort( [&] arr, left,  right: pivotIndex - 1);
80      quickSort( [&] arr,  left: pivotIndex + 1, right);
81  }
82
83  // STATIC chrono
84  std::vector<int> generate_static_vector(int size, unsigned int seed = 42) {
85      std::mt19937 rand(seed);//for the static output content
86      std::uniform_int_distribution<int> uniform_dist( a: 1,  b: size * 10);
87      std::vector<int> vec;
88      vec.reserve(size);
89      for (int i = 0; i < size; ++i) {
90          vec.push_back( x: uniform_dist( [&] rand));
91      }
92      return vec;
93  }
94
95
96  //DRIVER USING chono
97  void run_sorting_tests() {
98      std::vector<int> sizes = {100, 1000, 10000};
99
100     // Adjust sizes for standard C++ vector notation
101     std::vector<int> test_sizes = {100, 1000, 10000};
102
103     for (int size : test_sizes) {
104         // Generate static random data so both algorithms sort the exact same elements
105         std::vector<int> original_data = generate_static_vector(size,  seed: 42);
106
107         //Merge Sort
```

```
        std::vector<int> data_merge  = original_data;
        std::chrono::steady_clock::time_point start_merge = std::chrono::steady_clock::now();

        mergeSort( [&] data_merge,  left: 0,  right: size - 1);

        std::chrono::steady_clock::time_point end_merge = std::chrono::steady_clock::now();
        auto duration_merge :long long  = std::chrono::duration_cast<std::chrono::microseconds>( d: end_merge - start_merge).count();

        //Quick Sort
        std::vector<int> data_quick  = original_data;
        std::chrono::steady_clock::time_point start_quick = std::chrono::steady_clock::now();

        quickSort( [&] data_quick,  left: 0,  right: size - 1);

        std::chrono::steady_clock::time_point end_quick = std::chrono::steady_clock::now();
        auto duration_quick :long long  = std::chrono::duration_cast<std::chrono::microseconds>( d: end_quick - start_quick).count();

        // Output Results
        std::cout << "---------------------------------------\n";
        std::cout << "Input Size (N) = " << size << "\n";
        std::cout << "  * Merge Sort Time : " << duration_merge << " microseconds\n";
        std::cout << "  * Quick Sort Time : " << duration_quick << " microseconds\n";
    }
    std::cout << "---------------------------------------\n";
}

int main() {
    std::cout << "=== Sorting Algorithm Experimental Analysis ===\n";
    run_sorting_tests();
    return 0;
}
```

OUTPUT

```
C:\Users\royga\CLionProjects\untitled1\HOA14p1\HOA14p1.exe
Sorting Algorithm Experimental Analysis
---------------------------------------
Input Size (N) = 100
  * Merge Sort Time : 33 microseconds
  * Quick Sort Time : 5 microseconds
---------------------------------------
Input Size (N) = 1000
  * Merge Sort Time : 472 microseconds
  * Quick Sort Time : 71 microseconds
---------------------------------------
Input Size (N) = 10000
  * Merge Sort Time : 7144 microseconds
  * Quick Sort Time : 898 microseconds
---------------------------------------

Process finished with exit code 0
```

	Algorithm 1:Merge sort	Algorithm 2: Quick Sort
Best Case	**O(n log n)**	**O(n log n)**
Worst Case	**O (n log n)**	**O (n^2)**

ANALYSIS	
MENDOZA	Quick sort is much faster compared to merge sort, because merge sort takes time to divide every contents then to compared them to each other, meaning that higher number is causing more time and space to be used for it, unlike quick sort that runs on with like a 2 sub arrays, it is much faster and less resource extensive.
PILAPIL	Quick sort is faster than Merge sort because it sorts in place and uses less memory. Merge sort uses a temporary array to sort meanwhile Quick sort does it in-place giving an advantage in space.

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PSEUDOCODE

LINEAR

```
Algorithm LinearSearch(array, size, target)
   // Sequentially check each element from index 0 to size - 1
   For i = 0 to size - 1 do
      If array[i] == target then
         Return i  // Return the index where the element is found
      End If
   End For

   Return -1  // Return -1 if the element is not in the array
End Algorithm
```

BINARY

```
Algorithm BinarySearch(array, size, target)
   // Requires a pre-sorted array
   low = 0
   high = size - 1

   While low <= high do
      mid = low + (high - low) / 2  // Find the middle index
```

	If array[mid] == target then
	Return mid // Element found at index mid
	Else If array[mid] < target then
	low = mid + 1// Discard left half, search right
	Else
	high = mid - 1 // Discard right half, search left
	End If
	End While
	
	Return -1 // Return -1 if the element is not found
	End Algorithm

Pseudocode ANALYSIS

MENDOZA	Linear search uses a for loop implementation and it iterates until all of the content has been seen, while a binary search scans the contents by finding the midpoint at each step and halving the whole be halved at each step until the element that is being sought after is found
PILAPIL	Linear search works by visiting each index one by one until it finds the target value. On the other hand, Binary search works by repeatedly dividing the data, making it a faster searching algorithm compared to linear search.

MENDOZA	Linear search has a worst-case time complexity of O(n) because Linear search checks every element sequentially whereas binary search achieves O(log n) by cutting the search range in half. Both Linear search and binary search operate in-place requiring no extra memory overhead, which gives both Linear search and binary search the space complexity of O(1).
PILAPIL	Linear search's time complexity is O(n) because the loop iterates through all n elements and its space complexity is O(1). Binary Search's time complexity is O(log n) because the algorithm divides the search space in half and its space is also O(1).

Time and Space complexity Analysis

CODE

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>

// Linear Search
int linearSearch(const std::vector<int>& arr, int target) {
    for (size_t i = 0; i < arr.size(); ++i) {
        if (arr[i] == target) return i;
    }
    return -1;
}

// Binary Search
int binarySearch(const std::vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

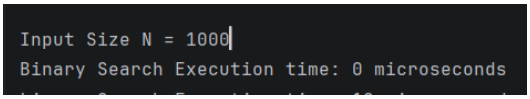
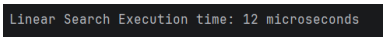
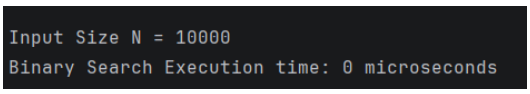
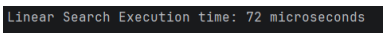
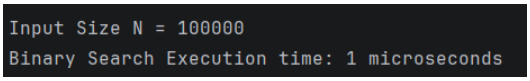
void benchmark(int n) {
    std::vector<int> arr(n);
    for (int i = 0; i < n; ++i) arr[i] = i * 2;
    int target = -1; // Worst-case: element does not exist

    auto startBS = std::chrono::high_resolution_clock::now();
    binarySearch(arr, target);
    auto endBS = std::chrono::high_resolution_clock::now();
    auto durationBS = std::chrono::duration_cast<std::chrono::microseconds>(endBS - startBS).count();

    // Benchmark Linear Search using microseconds
    auto startLS = std::chrono::high_resolution_clock::now();
    linearSearch(arr, target);
    auto endLS = std::chrono::high_resolution_clock::now();
    auto durationLS = std::chrono::duration_cast<std::chrono::microseconds>(endLS - startLS).count();

    std::cout << "Input Size N = " << n << "\n";
    std::cout << "Binary Search Execution time: " << durationBS << " microseconds\n";
    std::cout << "Linear Search Execution time: " << durationLS << " microseconds\n\n";
}

int main() {
    std::cout << "--- Worst-Case Benchmarking ---\n\n";
    benchmark(1000);
    benchmark(10000);
    benchmark(100000);
    return 0;
}
```

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000		
10,000		
100,000		

ANALYSIS

MENDOZA	The benchmark results show a difference in how linear search and binary search perform when the target is not in the list. Binary search is fast. It takes no time, mostly between zero and one microsecond no matter how big the list is. That's because it cuts the search space in half each time. On the other hand , linear search checks every item one by one so it gets slower as the list grows. For a list with 100,000 items it took 1,615 microseconds. These real results prove that binary search is much better, for searching sorted lists. It's just more efficient.
PILAPIL	The empirical performance data confirms that binary search achieves a significantly lower execution time than linear search. This efficiency stems from their algorithmic complexities, binary search follows a divide-and-conquer strategy O(log n), discarding half of the remaining elements at each step, while linear search evaluates each index sequentially O(n).

CONCLUSION

MENDOZA	Over all, this activity has taught us and made us remember the time and space complexity of the codes. Creating codes does not mean that all would be well, yes it can be executed but how it will be executed and how long is another talk, and this activity has enlightened me how to fix and other methods or processes to do.
PILAPIL	Through this Hands-on activity, we explored algorithmic complexity and analyzed differences between searching algorithms. Additionally, we conducted a side-by-side performance comparison of Quick Sort and Merge Sort. A place where I can improve is probably creating and analyzing pseudocode.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **None**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **None**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

